



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

计算机系统实验

第二次实验

CONTENTS

目录

「01」

实验目的

「02」

实验任务

「03」

实验测试

「04」

作业提交



➤ 实验背景

■ Shell在操作系统中的核心作用

- 命令行解释器：用户与内核交互的桥梁
- 功能范畴：进程创建、作业管理、I/O重定向、管道通信
- 典型Shell示例：**Bash**、**Zsh**、**Fish**

■ 技术依赖

- 系统调用：**fork**, **execve**, **waitpid**, **kill**等
- 信号机制：**SIGINT**, **SIGTSTP**, **SIGCHLD**
- UNIX环境：文件描述符、进程组、终端控制



➤ 实验目的

- **理解进程模型**：掌握 **fork** 创建子进程、**exec** 加载新程序的流程。
- **掌握信号机制**：处理 **SIGINT** (**Ctrl+C**)、**SIGTSTP** (**Ctrl+Z**) 等信号，实现作业控制。
- **实现 Shell 核心逻辑**：命令解析、进程管理、I/O 重定向的完整流程。
- **调试与测试**：通过单元测试与集成测试验证功能正确性。



- **实验文件下载地址（校内网）**：<https://file.cs-lab.top/s/BtJy7CRzHmmlpsQ>
- **学生实验数据包**：shlab-handout.tar
- **解压命令**：tar xvf shlab-handout.tar
- 数据包中主要包含下列文件：
 - **tsh.c**：tiny-shell 的代码框架，**要求实现里面的空函数**
 - **sdriver.pl**：用于模拟用户输入，以子进程的方式运行shell，并根据traceXX.txt轨迹文件向shell发送命令和信号。
 - **16个轨迹文件(trace file)**：trace01.txt trace16.txt
 - **tshref**：参考答案的程序（可执行文件），用于验证实验代码tsh.c的正确性：
 - **tiny-shell的输出应该与tshref的输出完全一致**



➤ 完成tsh.c中的空函数

tsh.c : tiny-shell 的代码框架，包含了基本的代码。

■ 在 *tsh.c* 文件的顶部注释处输入你的学号和姓名

■ 空函数是实验要求实现的部分

- ✓ **eval**: 解析和执行命令行的主程序 [70 lines]
- ✓ **builtin_cmd**: 识别以及执行内置 (built-in) 命令, 包括: **quit, fg, bg, jobs**. [25 lines]
- ✓ **do_bgfg**: 实现 bg 和 fg 内置命令. [50 lines]
- ✓ **waitfg**: 等待前台任务完成. [20 lines]
- ✓ **sigchld_handler**: 捕获 SIGCHLD 信号. 80 lines]
- ✓ **sigint_handler**: 捕获SIGINT (ctrl-c) 信号. [15 lines]
- ✓ **sigtstp_handler**: 捕获 SIGTSTP (ctrl-z) 信号. [15 lines]

■ 编译链接: unix> **make**

■ 运 行: unix> **./tsh**

tsh> [type commands to your shell here]



实验任务



➤ tsh模拟shell的行为，能够创建/回收进程、给进程发送信号等，应该具有以下特性：

- ① tsh的提示符： **tsh>**
- ② **命令行解析**：将用户输入的命令行字符串分割为独立的命令和参数。
- ③ 支持**4个内置命令**：
 - ✓ **quit**：退出终端；
 - ✓ **jobs**：列出所有后台作业；
 - ✓ **bg <job>**：后台作业，<job> 可以是 PID 或者 %JID 形式；
 - ✓ **fg <job>**：将一个处于运行或者停止状态的后台作业转移到前台继续运行。
- ④ 可以**执行外部程序**：对于非内置命令，使用fork()和exec()创建新进程执行。
- ⑤ **信号处理**：处理来自子进程或操作系统的信号，如SIGINT（中断信号）等。



➤ 进程控制模型

■ fork-exec模型

■ 进程组与会话：

- **setpgid(0, 0)**：子进程独立成组

➤ 信号处理机制

■ 信号传递规则：

- **Ctrl+C** → **SIGINT** → 终止前台进程组
- **Ctrl+Z** → **SIGTSTP** → 挂起前台进程组
- 子进程终止 → **SIGCHLD** → 回收僵尸进程

■ 信号处理函数设计：

- 异步安全函数限制（禁用**printf**等非可重入函数）
- 全局变量保护（**sigprocmask**屏蔽竞态代码）

```
// 外部命令处理
pid = fork();
if (pid == 0) { // 子进程
    execve();
} else {
    // 父进程：记录作业 → 等待前台作业
}
```

```
struct job {
    pid_t pgid;        // 进程组ID
    int jid;           // 作业ID
    char cmd[100];     // 命令字符串
    int state;         // 状态：FG（前台）/BG
                      // （后台）/ST（停止）
};
```

作业列表



实验任务分析：以trace01为例



- sdriver程序将读取trace01.txt，将该文件中的CLOSE和WAIT命令发送给shell子进程。
- CLOSE命令是发送EOF信号给shell，关闭shell，WAIT命令等待shell结束。

```
#  
# trace01.txt - Properly terminate on EOF.  
#  
CLOSE  
WAIT
```

在初始代码中已实现该功能（见tsh.c的main函数），因此不用修改代码也能通过该测试。

```
// tsh.c main()  
if (feof(stdin))  
{  
    /* End of file (ctrl-d) */  
    fflush(stdout);  
    exit(0);  
}
```



实验任务分析：以trace02为例



- 内置命令quit, 终止shell
- WAIT命令等待shell结束

```
#  
# trace02.txt - Process builtin quit command.  
#  
quit  
WAIT
```

shell执行一系列的**读/求值**步骤

- ① 读步骤读取用户的命令行
- ② 求值步骤解析命令, 代表用户运行

```
int main(){  
    char cmdline[MAXLINE]; /* command line */  
  
    while (1) {  
        /* read */  
        printf("> ");  
        Fgets(cmdline, MAXLINE, stdin);  
        if (feof(stdin))  
            exit(0);  
  
        /* evaluate */  
        eval(cmdline);  
    }  
}
```



实验任务分析：以trace02为例



参考CSAPP第8.4.6小节,
P525-526,

填写eval()和

builtin_command()框架

该框架是有缺陷的, 因为
它不回收后台子进程。后
面trace测试时再做修改。

```
void eval(char *cmdline)
{
    char *argv[MAXARGS]; /* Argument list execve() */
    char buf[MAXLINE]; /* Holds modified command line */
    int bg; /* Should the job run in bg or fg? */
    pid_t pid; /* Process id */

    strcpy(buf, cmdline);
    bg = parseline(buf, argv);
    if (argv[0] == NULL)
        return; /* Ignore empty lines */

    if (!builtin_command(argv)) {
        if ((pid = Fork()) == 0) { /* Child runs user job */
            if (execve(argv[0], argv, environ) < 0) {
                printf("%s: Command not found.\n", argv[0]);
                exit(0);
            }
        }

        /* Parent waits for foreground job to terminate */父进程等待前台子进程结束
        if (!bg) { //fg或bg或&
            int status;
            if (waitpid(pid, &status, 0) < 0)
                unix_error("waitfg: waitpid error");
        }
        else
            printf("%d %s", pid, cmdline);
    }
    return;
}
```

```
/* If first arg is a builtin command, run it and return true */
int builtin_command(char **argv)
{
    if (!strcmp(argv[0], "quit")) /* quit command */
        exit(0);
    if (!strcmp(argv[0], "&")) /* Ignore singleton & */
        return 1;
    return 0; /* Not a builtin command */
}
```



实验任务分析：以trace04为例



- echo -e表示打印转义字符，
\\046表示&。

```
#  
# trace04.txt - Run a background job.  
#  
/bin/echo -e tsh> ./myspin 1 \\046  
./myspin 1 &
```

- 运行myspin程序，使进程睡眠1秒，在后台执行myspin并输出：

jid pid 命令行

- 修改eval() bg的输出信息

```
void eval(char *cmdline)  
{  
  
    if (!bg) {        //fg或bg或&  
        int status;  
        if (waitpid(pid, &status, 0) < 0)  
            unix_error("waitfg: waitpid error");  
    }  
    else  
        printf("%d %s", pid, cmdline);  
}  
return;  
}
```



实验任务分析：以trace05为例



- 后台运行两个myspin进程，之后用jobs指令查看进程状态。
- **jobs**，显示所有通过tsh启动的进程及其当前状态。格式为：

[JID] (PID) 状态 对应的命令

```
#  
# trace05.txt - Process jobs builtin  
command.  
#  
/bin/echo -e tsh> ./myspin 2 \046  
./myspin 2 &  
  
/bin/echo -e tsh> ./myspin 3 \046  
./myspin 3 &  
  
/bin/echo tsh> jobs  
jobs
```

- ① 在builtin_command()添加jobs命令。
- ② 在eval()中添加add_job()，需要注意的特殊情况：（见CSAPP P537~539）
 - **子进程过早结束：信号阻塞，保证控制流的先后顺序**
 - **多个子进程同时结束：当tsh收到信号SIGCHLD，尽可能多的回收已经结束的子进程**



实验任务分析：以trace06为例



- 执行./mysplit 4，创建子进程并将其挂起 4 秒，而父进程在挂起 2 秒后发送 SIGINT 信号使子进程终止。

进程组：每一个进程都只属于一个进程组，一个进程组可以包含很多进程。默认的，一个子进程和它的父进程同属于一个进程组。

trace06.txt - 验证SIGINT转发至前台作业

/bin/echo -e "tsh> ./myspin 4" # 模拟用户输入

./myspin 4 # 前台运行4秒的

myspin SLEEP 2 # 等待2秒（确保前台作业正在运行）

INT # 发送SIGINT信号（模拟Ctrl+C）

- ① 在execve()之前，需要调用Setpgid()，设置新的进程组ID
- ② Kill(pid_t pid, int sig); // pid设置为 -(pid)，进程组的PID



实验测试



- **Shell driver**: 用shell驱动程序sdriver.pl模拟用户输入, 追踪测试文件Trace01~16.txt, 查看shell的输出。
- **参考程序**: 你的**shell**应该发出与**可执行文件tshref**参考解决方案相同的输出(当然, pid除外, 它会在每次运行时发生变化)。
- **测试方法1: 单用例测试**

- 测试你的shell程序tsh

unix> ./sdriver.pl -t trace01.txt -s ./tsh -a "-p"

或: unix> make test01

- 测试参考shell程序tshref

unix> ./sdriver.pl -t trace01.txt -s ./tshref -a "-p"

或: unix> make rtest01



实验测试



➤ 测试方法2：批量测试

unix> make clean & make # 如果有修改tsh.c, 记得先编译

unix> ./tsh_output.sh

➤ 可视化对比结果

➤ tsh.out

➤ tshref.out

```
tshref.out
1 ./sdriver.pl -t trace01.txt -s ./tsh -a "-p"
2 #
3 # trace01.txt - Properly terminate on EOF.
4 #
5 ./sdriver.pl -t trace02.txt -s ./tsh -a "-p"
6 #
7 # trace02.txt - Process builtin quit command.
8 #
9 ./sdriver.pl -t trace03.txt -s ./tsh -a "-p"
10 #
11 # trace03.txt - Run a foreground job.
12 #
13 tsh> quit
14 ./sdriver.pl -t trace04.txt -s ./tsh -a "-p"
15 #
16 # trace04.txt - Run a background job.
17 #
18 tsh> ./myspin 1 &
19 [1] (954709) ./myspin 1 &
20 ./sdriver.pl -t trace05.txt -s ./tsh -a "-p"
21 #

tsh.out
1+ make[1]: Entering directory `/afs/cs.cmu.edu/...
2 ./sdriver.pl -t trace01.txt -s ./tsh -a "-p"
3 #
4 # trace01.txt - Properly terminate on EOF.
5 #
6 ./sdriver.pl -t trace02.txt -s ./tsh -a "-p"
7 #
8 # trace02.txt - Process builtin quit command.
9 #
10 ./sdriver.pl -t trace03.txt -s ./tsh -a "-p"
11 #
12 # trace03.txt - Run a foreground job.
13 tsh> quit
14 ./sdriver.pl -t trace04.txt -s ./tsh -a "-p"
15 #
16 # trace04.txt - Run a background job.
17 #
18 tsh> ./myspin 1 &
19 [1] (26252) ./myspin 1 &
20+
21 ./sdriver.pl -t trace05.txt -s ./tsh -a "-p"
22 #
```



提交内容：实验报告（有模板）+ tsh.c文件（非压缩格式）

截止时间：

实验课后两周内提交至HITsz Grader 作业提交平台，具体截止日期参考平台发布。

- 登录网址：：<http://grader.tery.top:8000/#/login>
- 推荐浏览器：Chrome
- 初始用户名、密码均为学号，登录后请修改

注意

上传后可自行下载以确认是否正确提交



**同学们
请开始实验吧！**